요구사항 요약

환불 요청 중계 서버

쇼핑몰의 환불 요청을 받아 외부 결제대행사에 전달하고, 같은 환불이 두 번 처리되지 않도록 관리하는 서버를 만듭니다.

- 이 문서는 과제의 전체 흐름과 주요 요구사항을 정리한 **요약 자료**입니다.
- API 규격, 시나리오, 힌트 등 세부 내용은 과제 저장소의 `README.md`를 기준으로 확인해 주세요.
- 이 문서를 먼저 읽은 뒤 README를 처음부터 순서대로 읽는 것을 권장합니다.

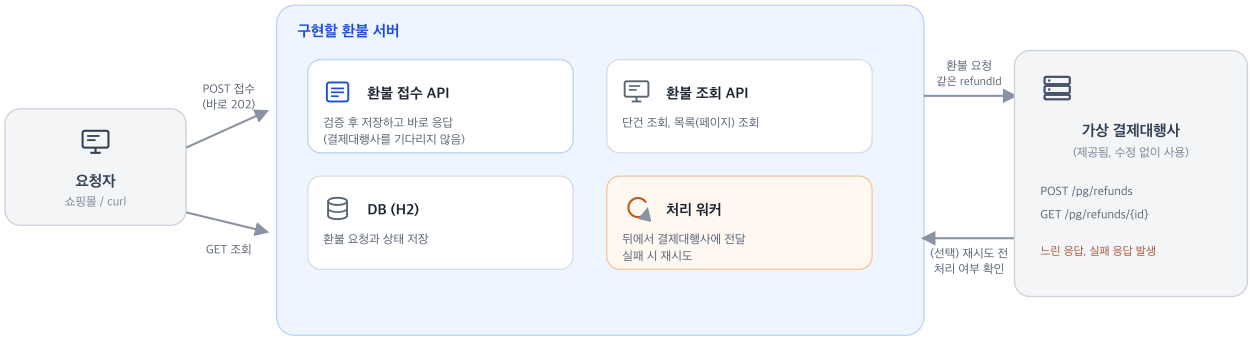
1개
구현할 서버
환불 요청 중계 서버 1개를 새로 구현합니다

2종
다룰 API
구현할 환불 서버의 API, 호출할 가상 결제대행사 API

13개
점검 시나리오
모든 시나리오가 기대 결과대로 동작해야 합니다

제한 없음
소요 시간
구현 속도보다 요구사항 충족 여부를 평가합니다

전체 구조



환불 접수 API(파란색)는 결제대행사의 응답을 기다리지 않고 바로 응답합니다. 결제대행사 호출은 처리 워커(주황색)가 별도로 담당합니다.

과제 목표와 구현 범위

과제 목표

- 환불 요청을 누락 없이 접수하고 **바로 응답**합니다.
- 결제대행사 호출이 실패하면 같은 refundId로 최대 4번까지 시도합니다.
- 같은 요청은 **한 번만 처리**합니다.
- 환불 한도, 상태, 시도 횟수와 실패 사유를 정확하게 관리합니다.

구현하지 않아도 되는 것

- 실제 결제 및 환불 연동 (가상 결제대행사로 대체)
- 로그인, 인증 및 인가
- Kafka 등 메시지 브로커 도입
- 대용량 트래픽 성능 튜닝

제공되는 가상 결제대행사

- 가상 결제대행사는 직접 구현하지 않습니다. Docker로 실행해 사용하는 **모의(mock) 서버**입니다.
- 실제 외부 시스템에서 생길 수 있는 상황을 연습할 수 있도록, 아래 세 가지 동작을 의도적으로 재현합니다.

느린 응답
응답이 몇 초 이상 걸릴 수 있습니다. 접수 API는 이 응답을 기다리면 안 됩니다.

실패 및 오류 응답
일정 확률로 실패나 내부 오류(5xx)를 응답합니다. 재시도 로직이 필요합니다.

같은 환불 ID는 한 번만 처리
이미 성공한 환불 ID로 다시 요청하면 환불하지 않고 이전 결과를 돌려줍니다. 이중 환불을 막을 때 활용합니다.

가장 놓치기 쉬운 상황: 응답은 늦었지만 환불은 이미 처리된 경우

- 환불 서버가 응답 대기 시간을 넘겨 **요청을 실패로 판단**했지만, 결제대행사는 이미 환불을 처리했을 수 있습니다.
- 이때 새로운 요청 ID로 재시도하면 **환불이 두 번** 처리됩니다.

1. 요청
처리 워커가 결제대행사에 환불을 요청합니다.

→

2. 시간 초과
응답이 늦어 실패로 판단하고 재시도를 준비합니다. 실제로는 환불이 처리되었습니다.

→

3. 같은 ID로 재시도
결제대행사가 이미 처리된 요청이라고 응답합니다.

→

4. 결과 반영
새 refundId로 다시 요청하지 않고, 결제대행사가 돌려준 성공 결과대로 상태를 COMPLETED로 바꿉니다.

자세한 시퀀스와 API 응답 형식은 README.md 6장을 참고해 주세요.

반드시 지켜야 할 API 명세

항목	규칙
환불 접수	`POST /api/v1/refunds` 정상 접수 `202`, 같은 ID와 같은 내용 재요청 `200`, 같은 ID와 다른 내용 `409`, 한도 초과 `422`, 입력 오류 `400`
환불 조회	`GET /api/v1/refunds`, `GET /api/v1/refunds/{refundId}` 최신순 정렬, 페이지 처리
상태	`ACCEPTED`, `COMPLETED`, `FAILED`, `REJECTED` API 응답에는 이 4가지 상태만 사용합니다.
재시도	최초 시도를 포함해 **최대 4번** 시도하며, 재시도할 때도 같은 `refundId`를 사용합니다.
결제대행사	`POST /pg/refunds`, `GET /pg/refunds/{refundId}` 제공된 API를 수정 없이 사용합니다.

점검 시나리오 13개

1. 접수 단계 (S1~S7) 7개

- 정상 접수: 바로 202 응답
- 같은 ID, 같은 내용으로 재요청: 200 응답, 기존 결과 반환
- 같은 ID, 다른 내용으로 요청: 409 응답
- 같은 ID로 동시에 여러 건 요청: 정확히 1건만 저장
- 접수된 금액과 이번 금액의 합이 한도 초과: 422 응답, REJECTED 상태로 기록
- 같은 주문에 여러 건 동시 요청: 한도 안에서만 접수
- 금액이 0 이하이거나 필수값 누락: 400 응답

2. 처리 단계 (S8~S12) 5개

- 결제대행사 성공 응답: COMPLETED
- 실패 또는 오류 응답: 재시도 후에도 실패하면 FAILED
- 응답 대기 시간 초과: 실패로 보고 재시도
- 같은 주문의 환불: 접수 순서대로 한 건씩 처리
- 응답은 늦었지만 실제로는 처리된 경우: 환불은 **정확히 1번**만 처리

3. 조회 (S13) 1개

- 주문별 목록 조회: 최신순 정렬, 페이지 처리

평가 배점 100점

- 기능 정확성 35점: 시나리오 S1~S13
- 설계 품질 35점
- 설계 문서 20점
- 코드 품질 10점

권장 진행 순서

1. **이 문서로 과제의 전체 흐름을 확인합니다.**
   구현할 서버와 제공되는 자료를 먼저 확인해 주세요.
2. **README.md를 처음부터 끝까지 읽습니다.**
   API 형식, 에러 코드, 시나리오 13개가 모두 정리되어 있습니다.
3. **`docker compose up` 으로 가상 결제대행사를 실행합니다.**
   환불 서버보다 먼저 실행하고, 결제대행사 Swagger(`localhost:9090/swagger-ui.html`)에서 API 동작을 확인해 주세요.
4. **`starter-project` 에서 구현을 시작합니다.**
   정상 접수(S1)부터 구현하고, 시나리오를 하나씩 추가해 나가는 것을 권장합니다.
5. **구현이 막히면 README 11장의 힌트를 하나씩 확인합니다.**
   힌트는 정답 대신 고민할 방향을 알려 줍니다.

정해진 정답이 있는 과제가 아닙니다. 요구사항을 충족하는 범위에서 설계를 선택하고, 선택한 이유를 설계 문서에 작성해 주세요.
세부 요구사항은 과제 저장소의 **README.md**를 기준으로 확인해 주세요.